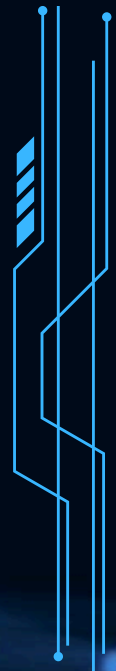
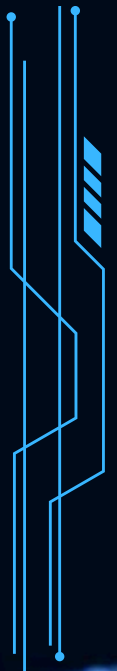


Kevin Jiménez Escalante
2024253395
01 de septiembre de 2026



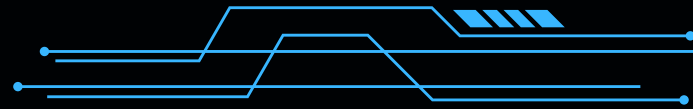
APUNTES DE COMPILADORES E INTÉRPRETES



PRIMOS DE LOS COMPILADORES

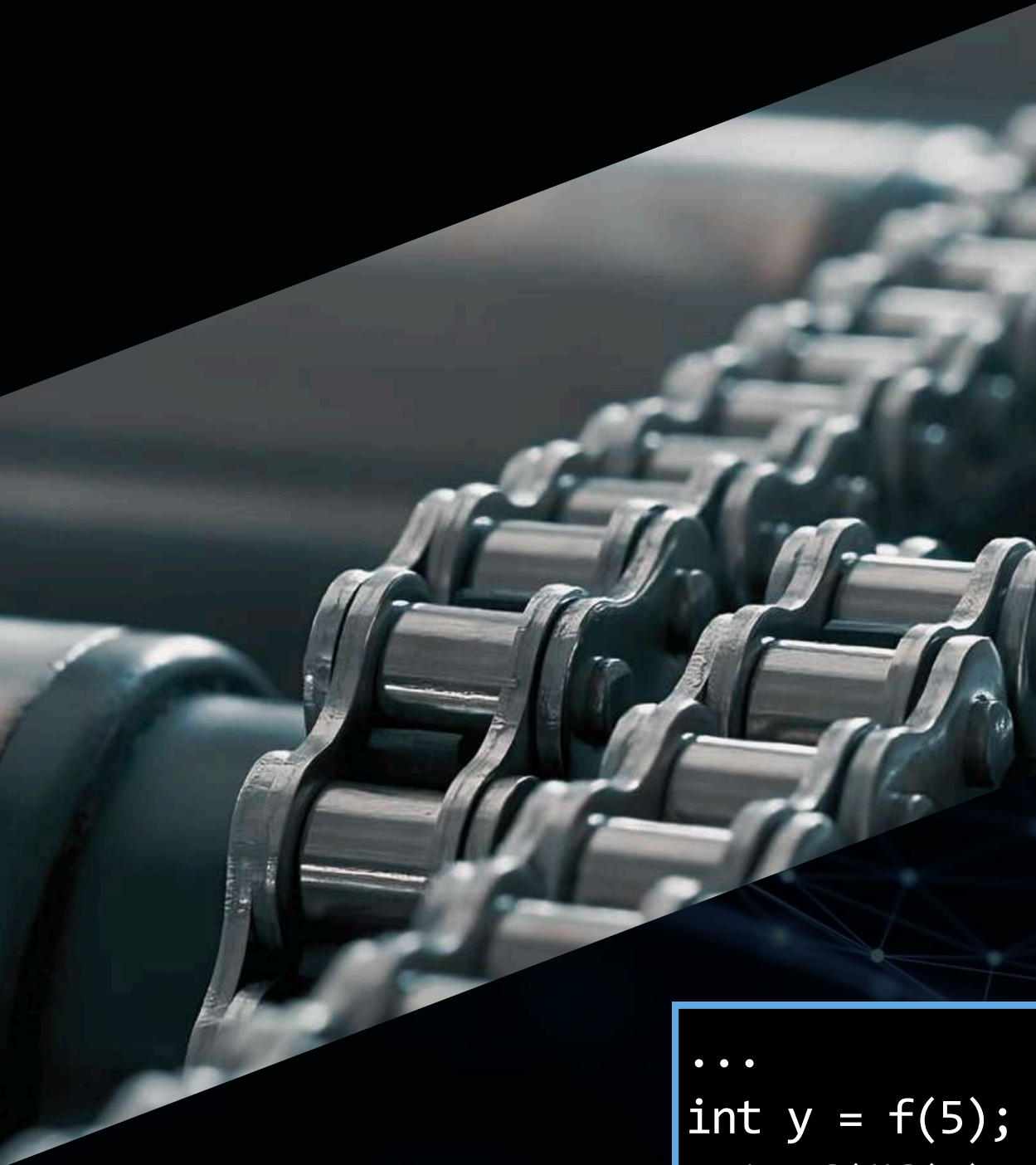
- Programas relacionales
- Ensambladores
- Desensambladores
- Intérpretes
- Preprocesadores
- **Linker**
- **Loader**
- **Debugger**
- **Profiler**
- **Editor de texto**
- **Manejador de versiones**





LINKER

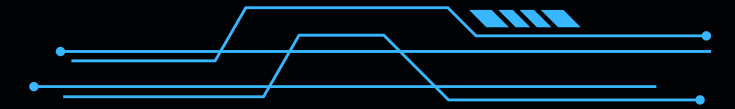
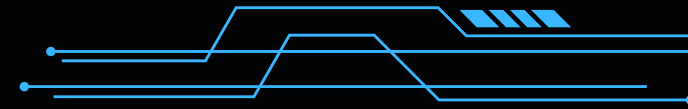
Normalmente, los proyectos no son un único archivo, contienen varios archivos objeto que interactúan entre sí. A partir de esto hay que generar un único ejecutable.



```
...  
int y = f(5);  
printf("f(5) = %d", y);  
...
```

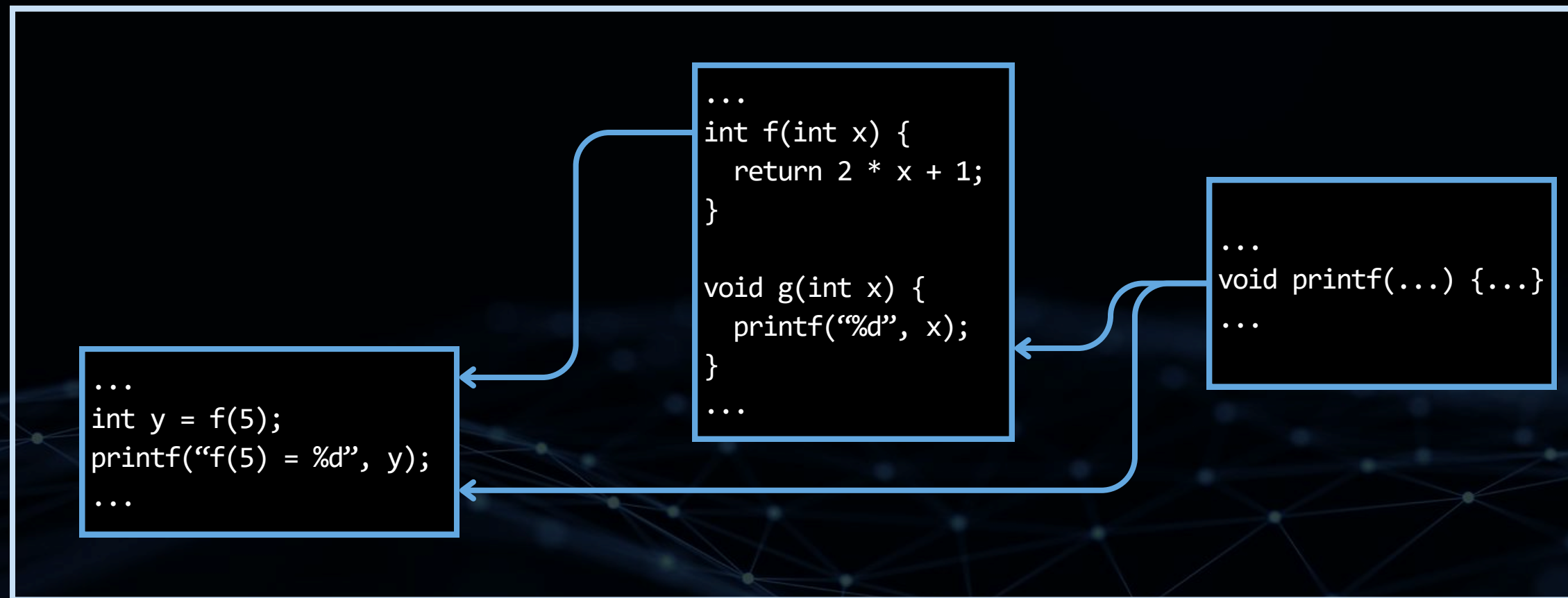
```
...  
int f(int x) {  
    return 2 * x + 1;  
}  
  
void g(int x) {  
    printf("%d", x);  
}  
...
```

```
...  
void printf(...) {...}  
...
```

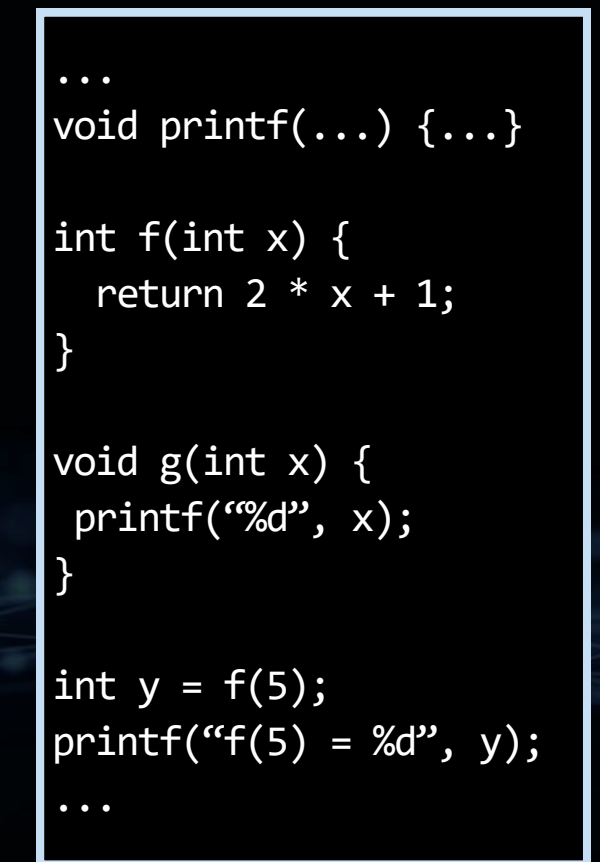


Cuando el linker encuentra una funcionalidad no declarada, busca en las bibliotecas incluidas e inserta la definición en el ejecutable. En Linux se llama ld.

Archivos objeto (definitivamente es C)



Ejecutable



¿LINKING ESTÁTICO O DINÁMICO?



LINKING DINAMICO

Agarra todos las bibliotecas y programas necesarios para formar un sólo ejecutable completo que se pueda correr en cualquier momento.

- Es más rápido
- Mejor portabilidad, es un programa independiente
- Siempre va a “funcionar igual”
- Si hay que actualizar algo, hay que actualizar todo

LINKING ESTÁTICO

Se genera un ejecutable más pequeño porque no incluye cada función necesaria. Si no encuentra una función, el **handler** la busca dinámicamente, la pega en el programa y ejecuta la función. Esto pasa sólo si hay que ejecutarla, no durante tiempo de compilación.

- El ejecutable es más pequeño
- Más lento (overhead al llamar una función externa por primera vez)
- Hay que tener las dependencias
- Si hay que actualizar las dependencias, no hay que “linkear” todo de nuevo
- El programa puede dejar de funcionar porque una de sus dependencias tiene problemas

¿SE PUEDEN LINKEAR DIFERENTES LENGUAJES?

Al final de cuentas, son todos archivos objeto, ¿no?

Esto funciona sólo si se siguen las “calling conventions” entre el caller y el callee. Esto se refiere a que los lenguajes tienen la misma forma de dar y recibir parámetros en los procedimientos para obtener el resultado esperado.

```
vec<T> new();

for (int i in input {
    output.push(element * 2);
}

output

}

fn vec_map_example(input: &[i32])
    input.iter().map(|element| element * 2)
}

fn vec_map(input: &[i32])
    input.iter().map(|element| element * 2)
}

// ...

BSTNode<E>* root;

BSTNode<E>* insertAux(BSTNode<E>* current, E element)
    if (current == nullptr)
        return new BSTNode<E>(element);
    // En caso de que no acepte repetidos
    if (element == current->element)
        throw runtime_error("No repetidos");
    if (element < current->element) {
        current->left = insertAux(current->left, element);
    }
    else {
        current->right = insertAux(current->right, element);
    }
    return current;
}

boolean ...
```

LOADER

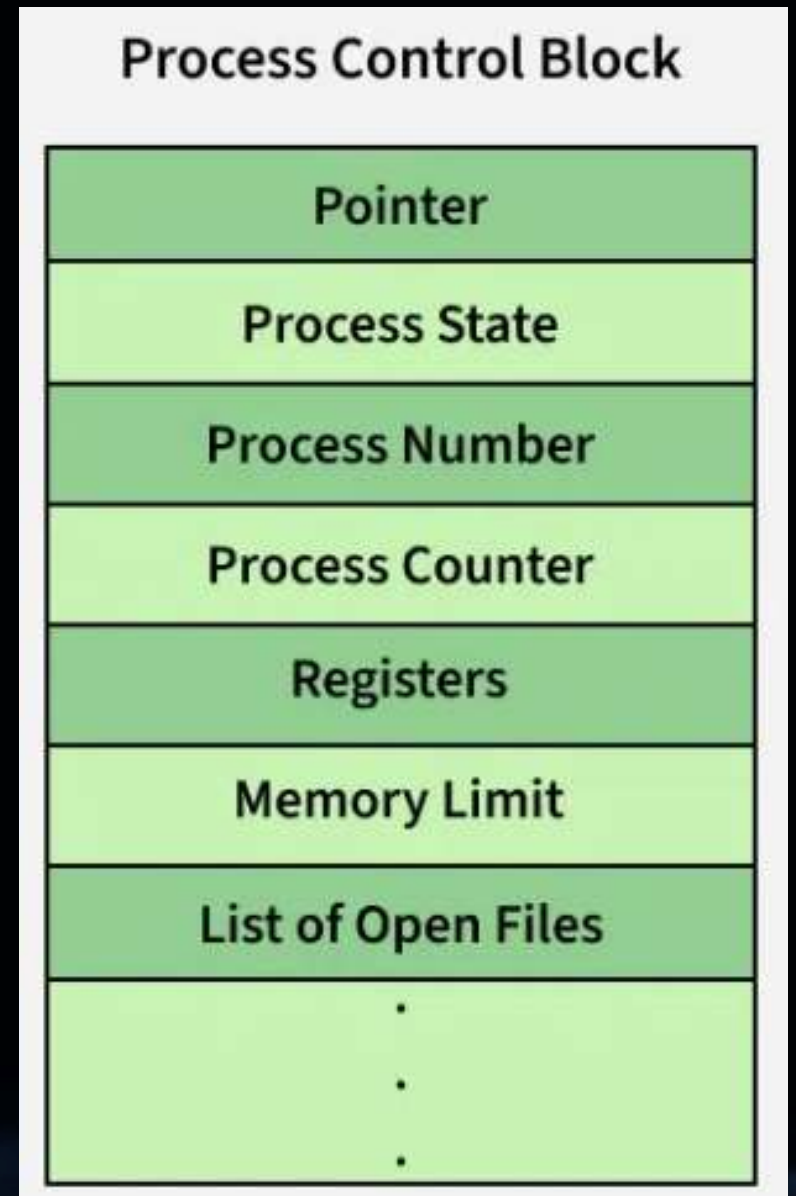
Un proceso es un programa en ejecución. El loader es un módulo del sistema operativo que se encarga de convertir el programa en un proceso. Necesita información del compilador y del linker.

El SO llama al loader para las siguientes tareas:

- Leer el ejecutable
- Negociar y administrar memoria
- Convertir en un proceso mediante la creación de un PCB. Un PCB (Process control block), es como la identificación, es un ente “schedulable” o agendable que se puede agendar y administrar.

Una funcionalidad muy importante del loader es la relocalización.

Dato curioso: Hay procesos no “schedulables”.



RELOCALIZACIÓN

Los ejecutables normalmente tienen referencias a memoria para saltar a otras partes del programa. El problema es que estas referencias de memoria son respecto al inicio del programa, pero se deben de cambiar (relocalizar) para que el programa funcione de manera correcta y no pueda acceder a partes de memoria a las que no debería.

No todo es sensible a la posición, la asignación de valores y shift no lo son, pero los jumps y calls sí. Entonces, ¿cómo hacemos para saber que movemos y que no?

El compilador crea una tabla de relocalización que dice cuales referencias son sensibles a la posición. Esta tabla es un bitmap donde cada bit representa si un byte tiene que ser relocalizado.

RELOCALIZACIÓN

Aquí hay un ejemplo muy simplificado de porque relocalizar es importante, si no se relocaliza, se toma un ejecutable, se pone en memoria y la compu se convierte en un pisapapeles premium.

Memoria

Dirección A: Alguna función del S0
Dirección B: Otra función del S0
Dirección C: Algo de I/O
Dirección D: Elimine System 32

Ejecutable

Dirección A: Alguna función del programa
Dirección B: Otra función del programa
Dirección C: Alguna otra función del programa
Dirección D: Una última función del programa

Llame a función A
...
Llame a función D
...
Llame a función B



Memoria

Dirección A: Alguna función del S0
Dirección B: Otra función del S0
Dirección C: Algo de I/O
Dirección D: Elimine System 32

Dirección E: Alguna función del programa
Dirección F: Otra función del programa
Dirección G: Alguna otra función del programa
Dirección H: Una última función del programa

Llame a función A
...
Llame a función D
...
Llame a función B



RELOCALIZACIÓN

Al relocalizar, el loader usa la tabla de relocalización para mover las referencias sensibles a la posición al lugar correspondiente.

Memoria

Dirección A: Alguna función del S0
Dirección B: Otra función del S0
Dirección C: Algo de I/O
Dirección D: Elimine System 32

Ejecutable

Dirección A: Alguna función del programa
Dirección B: Otra función del programa
Dirección C: Alguna otra función del programa
Dirección D: Una última función del programa

Llame a función A
...
Llame a función D
...
Llame a función B



Memoria

Dirección A: Alguna función del S0
Dirección B: Otra función del S0
Dirección C: Algo de I/O
Dirección D: Elimine System 32

Dirección E: Alguna función del programa
Dirección F: Otra función del programa
Dirección G: Alguna otra función del programa
Dirección H: Una última función del programa

Llame a función E
...
Llame a función H
...
Llame a función F

DEBUGGER

¿¡Magia!? Osea, ¿Cómo hace para controlar la ejecución de un programa o de ver el contenido de variables? ¿¡Existe otra forma de arreglar el código sin poner prints cada 2 líneas!?

La forma de controlar la ejecución del programa con breakpoints (al menos antes) es poniendo una llamada al debugger en el programa.

Para ver el contenido de las variables, hay que preguntarle al compilador. Linux tiene opciones para debugging que le dicen al compilador que guarde información para que el debugger la use. Relaciona el código fuente con el LM usando la tabla de símbolos.

Ahora hay características de debugging en los procesadores porque es algo que se hacía mucho, algo similar pasó con las instrucciones de PUSH y POP.



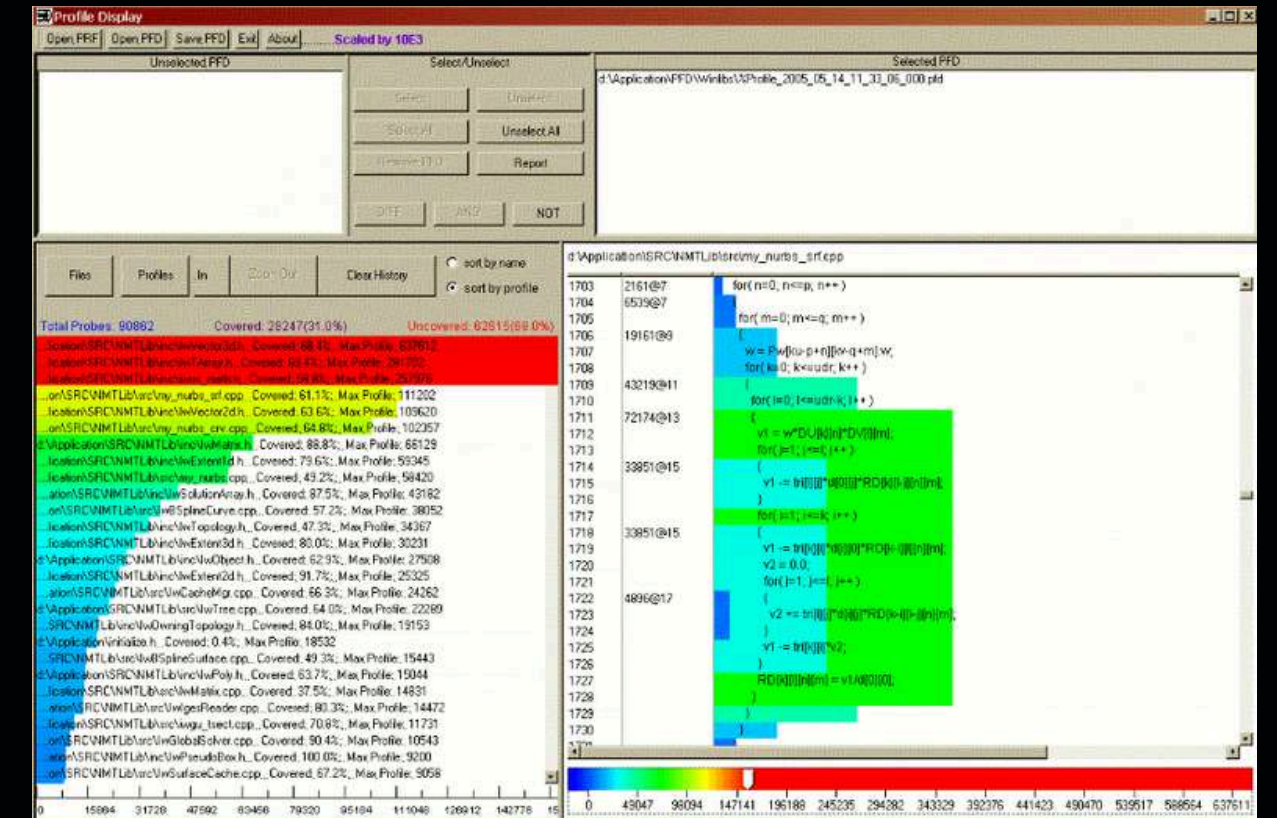
PROFILER

Herramienta que extrae estadísticas acerca del tiempo de ejecución del código.

Usa el PC (program counter) para saber en cuál parte del programa actualmente y la tabla de símbolos para saber a cuál procedimiento corresponde la posición del PC.

Cada cierta cantidad de tiempo, el profiler revisa dónde se encuentra el PC, con esta información se pueden crear estadísticas de los lugares que representan mayor porcentaje del tiempo de ejecución entre otros.

Esto es útil para prestar atención a mejorar las partes del programa que signifiquen mayor parte del tiempo de ejecución.

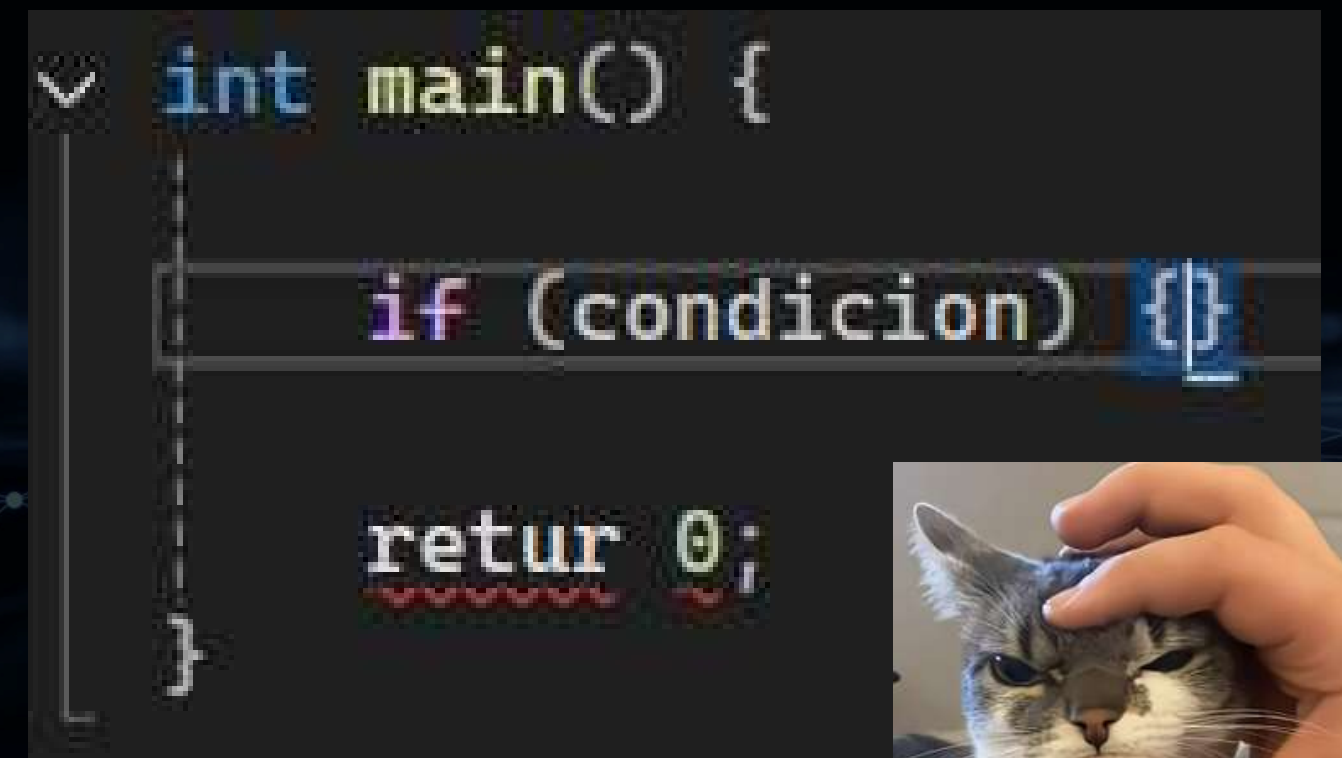
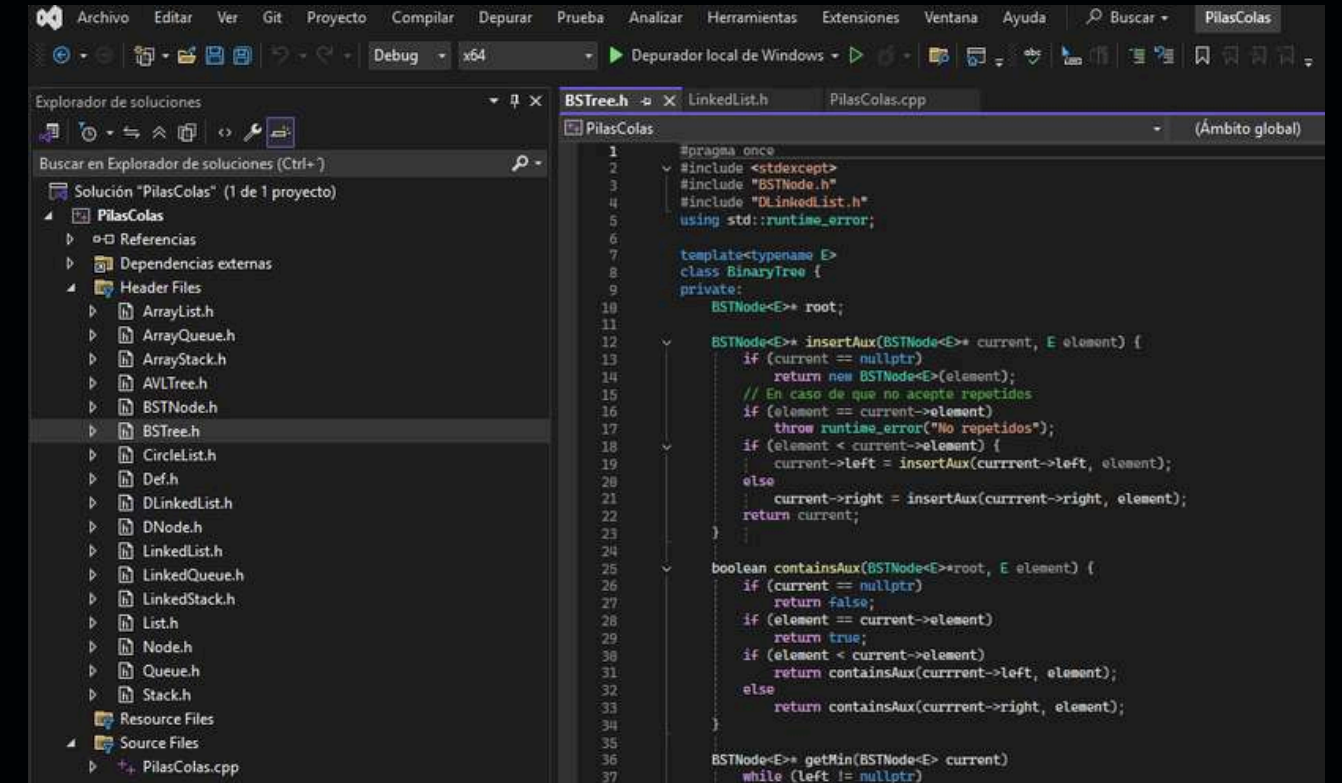


EDITOR DE TEXTO (IDE)

Los programadores son bien malcriados y lo quieren todo fácil.

Los IDE (Integrated development environment) son herramientas útiles para crear y modificar archivos fuente.

Tienen algunas funcionalidades de compiladores como escáner y parser que entienden el lenguaje y ayudan con el desarrollo. En este ejemplo colorea el texto, autocompleta la llave del condicional y advierte sobre un error.



MANEJADOR DE VERSIONES

Git fué creado por Linus Torvald :O

En proyectos grandes hay mucha gente trabajando, creando, modificando y borrando archivos. Un manejador de versiones sirve para documentar autores, tener un historial de versiones, crear y mezclar ramas y mucho más.

Lo mismo

author authored on May 12, 2025

Puse un archivo en un lugar equivocado ups

author authored on May 12, 2025

Docu pal javaDoc

author authored on May 12, 2025

Cambié el formato de los archivos

author authored on May 12, 2025

Lo del excel esta completo por fin ...

author authored on May 12, 2025

no fue perron este codigo Delete Lab9/Punto1.sql

author authored on Nov 23, 2025

Codigo perron de lab 9

author authored on Nov 23, 2025

GRACIAS
POR VER